Laboratorio 5 - Visión Por Computadora

Autores:

- Nelson García
- Joaquín Puente
- Diego Linares

Task 1

In [153…

```
# Importar librerías

import cv2
import numpy as np
import matplotlib.pyplot as plt
```

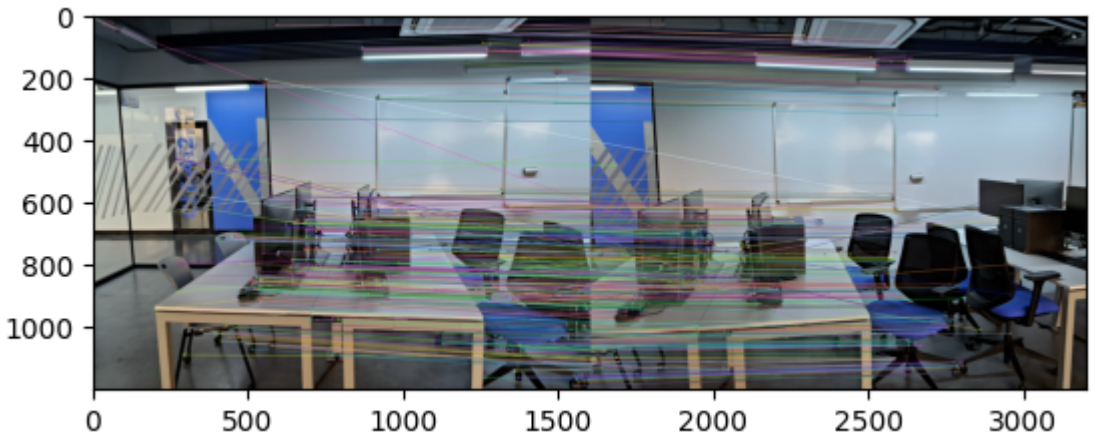
Set de control:

In [154…

```
#Carga de imagenes
img_left = cv2.imread('dataset/control/izq.jpeg')
img_right = cv2.imread('dataset/control/derecha.jpeg')

#Convertir en grises

gray_left = cv2.cvtColor(img_left, cv2.COLOR_BGR2GRAY)
gray_right = cv2.cvtColor(img_right, cv2.COLOR_BGR2GRAY)

# SIFT para detectar keypoints

sift = cv2.SIFT_create()
kp_left, des_left = sift.detectAndCompute(gray_left, None)
kp_right, des_right = sift.detectAndCompute(gray_right, None)

#Matching flann puntos similares enttre imagenes

FLANN_INDEX_KDTREE = 1
index_params = dict(algorithm = FLANN_INDEX_KDTREE, trees = 5)
search_params = dict(checks = 50)
flann = cv2.FlannBasedMatcher(index_params, search_params)

#Derecha es query

#Izquierda es nuestro train

matches = flann.knnMatch(des_left, des_right, k=2)

good_matches = []

for m,n in matches:
    if m.distance < 0.7*n.distance:
        good_matches.append(m)

print(len(good_matches))
```

```
#Visualizar

img3 = cv2.drawMatches(img_left, kp_left, img_right, kp_right, good_matches, Non
plt.imshow(img3)
plt.show()
```

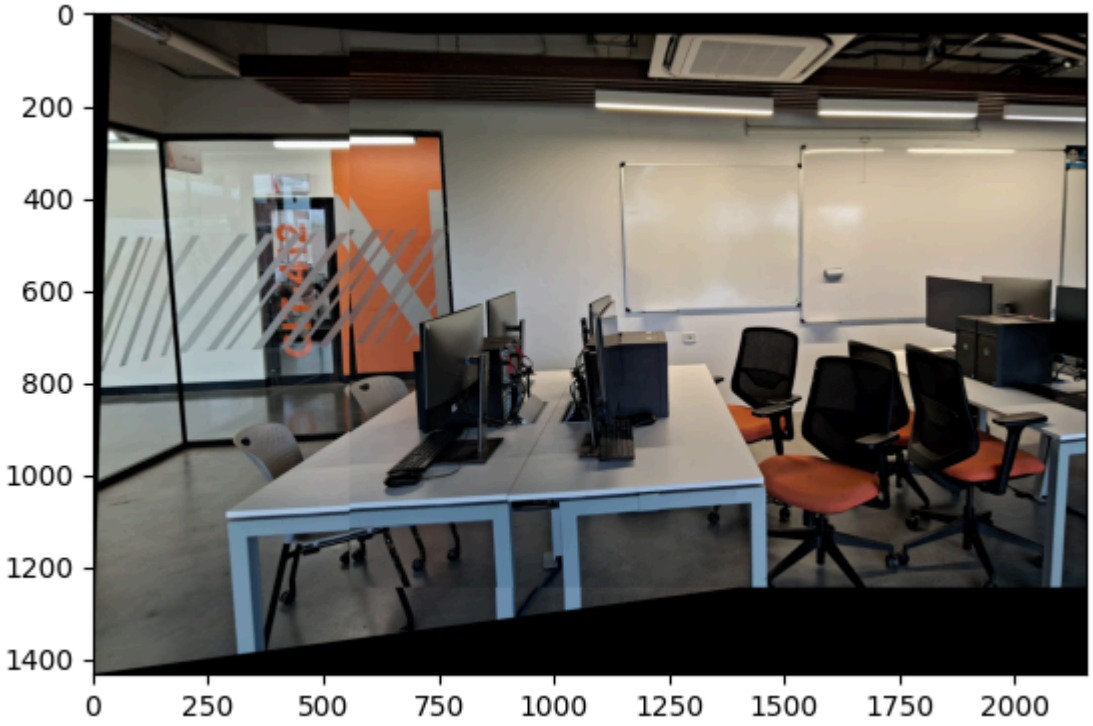
```
450
```



In [155…

```
#Homografia

#Extraer coordenadas de los x, y
src_pts = np.float32([ kp_left[m.queryIdx].pt for m in good_matches ]).reshape(-
dst_pts = np.float32([ kp_right[m.trainIdx].pt for m in good_matches ]).reshape(

#RANSAC
M, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)

#Calcular la altura y anchura de ambas imágenes
h_left, w_left = img_left.shape[:2]
h_right, w_right = img_right.shape[:2]

#Transformar las esquinas de la imagen izquierda con la homografía M
pts_left = np.float32([[0,0],[0,h_left],[w_left,h_left],[w_left,0]]).reshape(-1,
pts_left_transformed = cv2.perspectiveTransform(pts_left, M)

#Definir las esquinas de la imagen derecha
pts_right = np.float32([[0,0],[0,h_right],[w_right,h_right],[w_right,0]]).reshap

#Unir todos los puntos para encontrar el bounding box global
all_pts = np.concatenate((pts_left_transformed, pts_right), axis=0)

#Encontrar los límites mínimo y máximo de x e y
x_min, y_min = np.int32(all_pts.min(axis=0).ravel() - 0.5)
x_max, y_max = np.int32(all_pts.max(axis=0).ravel() + 0.5)

#Crear una matriz de traslación para mover el panorama a coordenadas positivas
T = np.array([[1, 0, -x_min],
              [0, 1, -y_min],
              [0, 0, 1]], dtype=np.float32)

#Calcular las nuevas dimensiones del panorama
panorama_width = x_max - x_min
panorama_height = y_max - y_min

#Crear el panorama y deformar la imagen izquierda con la homografía ajustada
```

```
panorama = cv2.warpPerspective(img_left, T @ M, (panorama_width, panorama_height

mask_right = np.zeros((panorama_height, panorama_width, 3), dtype=np.uint8)
mask_right[-y_min:-y_min+h_right, -x_min:-x_min+w_right] = img_right


for i in range(h_right):
    for j in range(w_right):
        if np.any(img_right[i, j] != [0, 0, 0]):
            if -y_min + i < panorama_height and -x_min + j < panorama_width:
                panorama[-y_min + i, -x_min + j] = img_right[i, j]


plt.imshow(cv2.cvtColor(panorama, cv2.COLOR_BGR2RGB))
plt.show()
```



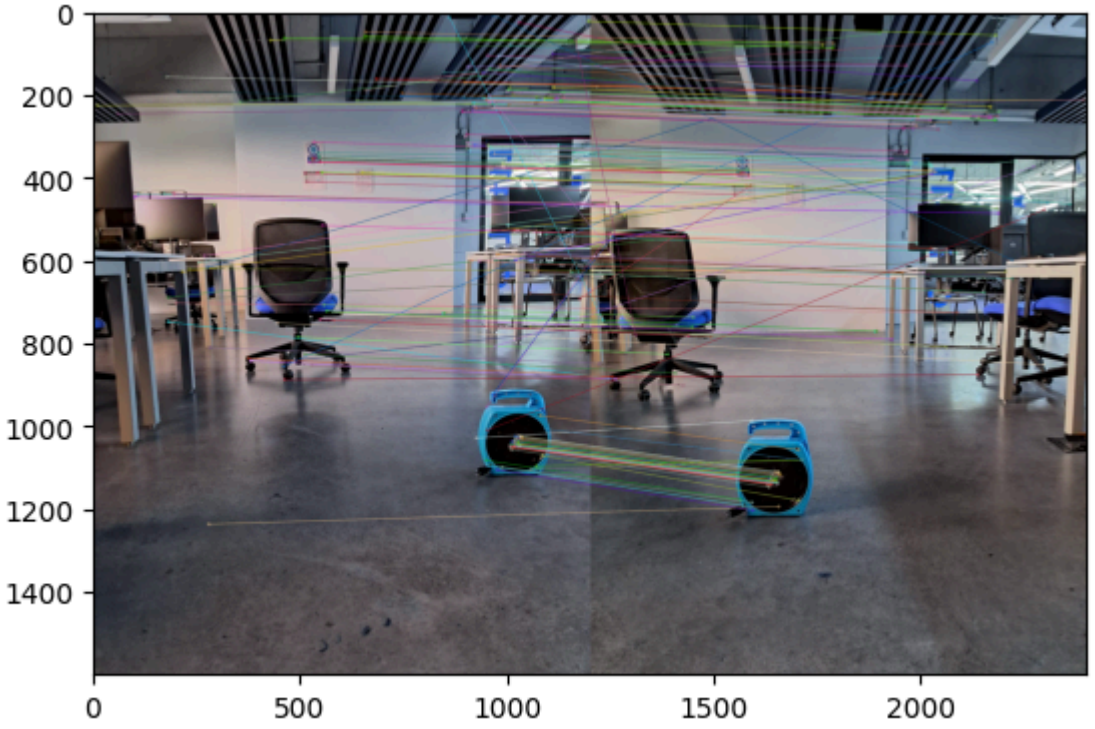
Set experimental:

In [156…

```
#Carga de imagenes
img_left = cv2.imread('dataset/experimental/izq_exp.jpeg')
img_right = cv2.imread('dataset/experimental/derecha_exp.jpeg')

#Convertir en grises

gray_left = cv2.cvtColor(img_left, cv2.COLOR_BGR2GRAY)
gray_right = cv2.cvtColor(img_right, cv2.COLOR_BGR2GRAY)

# SIFT para detectar keypoints

sift = cv2.SIFT_create()
kp_left, des_left = sift.detectAndCompute(gray_left, None)
kp_right, des_right = sift.detectAndCompute(gray_right, None)

#Matching flann puntos similares enttre imagenes

FLANN_INDEX_KDTREE = 1
index_params = dict(algorithm = FLANN_INDEX_KDTREE, trees = 5)
```

```
search_params = dict(checks = 50)
flann = cv2.FlannBasedMatcher(index_params, search_params)

#Derecha es query

#Izquierda es nuestro train

matches = flann.knnMatch(des_left, des_right, k=2)

good_matches = []

for m,n in matches:
    if m.distance < 0.7*n.distance:
        good_matches.append(m)

print(len(good_matches))

#Visualizar

img3 = cv2.drawMatches(img_left, kp_left, img_right, kp_right, good_matches, Non
plt.imshow(img3)
plt.show()
```

```
206
```



In [157…

```
#Homografia

#Extraer coordenadas de los x, y
src_pts = np.float32([ kp_left[m.queryIdx].pt for m in good_matches ]).reshape(-
dst_pts = np.float32([ kp_right[m.trainIdx].pt for m in good_matches ]).reshape(

#RANSAC
M, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)

#Calcular la altura y anchura de ambas imágenes
h_left, w_left = img_left.shape[:2]
h_right, w_right = img_right.shape[:2]
```

```
#Transformar las esquinas de la imagen izquierda con la homografía M
pts_left = np.float32([[0,0],[0,h_left],[w_left,h_left],[w_left,0]]).reshape(-1,
pts_left_transformed = cv2.perspectiveTransform(pts_left, M)

#Definir las esquinas de la imagen derecha
pts_right = np.float32([[0,0],[0,h_right],[w_right,h_right],[w_right,0]]).reshap

#Unir todos los puntos para encontrar el bounding box global
all_pts = np.concatenate((pts_left_transformed, pts_right), axis=0)

#Encontrar los límites mínimo y máximo de x e y
x_min, y_min = np.int32(all_pts.min(axis=0).ravel() - 0.5)
x_max, y_max = np.int32(all_pts.max(axis=0).ravel() + 0.5)

#Crear una matriz de traslación para mover el panorama a coordenadas positivas
T = np.array([[1, 0, -x_min],
              [0, 1, -y_min],
              [0, 0, 1]], dtype=np.float32)

#Calcular las nuevas dimensiones del panorama
panorama_width = x_max - x_min
panorama_height = y_max - y_min

#Crear el panorama y deformar la imagen izquierda con la homografía ajustada
panorama = cv2.warpPerspective(img_left, T @ M, (panorama_width, panorama_height

mask_right = np.zeros((panorama_height, panorama_width, 3), dtype=np.uint8)
mask_right[-y_min:-y_min+h_right, -x_min:-x_min+w_right] = img_right


for i in range(h_right):
    for j in range(w_right):
        if np.any(img_right[i, j] != [0, 0, 0]):
            if -y_min + i < panorama_height and -x_min + j < panorama_width:
                panorama[-y_min + i, -x_min + j] = img_right[i, j]


plt.imshow(cv2.cvtColor(panorama, cv2.COLOR_BGR2RGB))
plt.show()
```

Task 2

Task 2.1 – Visualización del Error

In [158…

```
left_on_canvas = cv2.warpPerspective(img_left, T @ M, (panorama_width, panorama_

right_on_canvas = np.zeros_like(left_on_canvas)
y0, x0 = -y_min, -x_min
right_on_canvas[y0:y0+h_right, x0:x0+w_right] = img_right

left_gray = cv2.cvtColor(left_on_canvas, cv2.COLOR_BGR2GRAY).astype(np.float32)
right_gray = cv2.cvtColor(right_on_canvas, cv2.COLOR_BGR2GRAY).astype(np.float32

mask_left = cv2.cvtColor(left_on_canvas, cv2.COLOR_BGR2GRAY) > 0
mask_right = cv2.cvtColor(right_on_canvas, cv2.COLOR_BGR2GRAY) > 0
overlap = mask_left & mask_right

ys, xs = np.where(overlap)
r_min, r_max = ys.min(), ys.max()
c_min, c_max = xs.min(), xs.max()

left_crop = left_gray[r_min:r_max+1, c_min:c_max+1]
right_crop = right_gray[r_min:r_max+1, c_min:c_max+1]

anaglyph = np.zeros((*left_crop.shape, 3), dtype=np.float32)
anaglyph[..., 0] = left_crop
anaglyph[..., 1] = right_crop
anaglyph[..., 2] = right_crop

anaglyph_u8 = np.clip(anaglyph * 255.0, 0, 255).astype(np.uint8)
```

```
plt.figure(figsize=(14, 7))
plt.title("Anaglifo Rojo/Cian (alineado = gris, error = bordes de color)")
plt.imshow(anaglyph_u8)
plt.axis("off")
plt.show()
```

Anaglifo Rojo/Cian (alineado = gris, error = bordes de color)

Task 2.2 – Visualización del Error

In [159…

```
anaglyph_rgb = cv2.cvtColor(anaglyph_u8, cv2.COLOR_BGR2RGB)

red_ghost = np.zeros_like(anaglyph_rgb)
red_ghost[..., 0] = anaglyph_rgb[..., 0]

cyan_ghost = np.zeros_like(anaglyph_rgb)
cyan_ghost[..., 1] = anaglyph_rgb[..., 1]
cyan_ghost[..., 2] = anaglyph_rgb[..., 2]

fig, axes = plt.subplots(1, 3, figsize=(20, 7))
axes[0].imshow(anaglyph_rgb); axes[0].set_title("Anaglifo completo")
axes[1].imshow(red_ghost);    axes[1].set_title("Fantasma 1 (Rojo – img izquierd
axes[2].imshow(cyan_ghost);   axes[2].set_title("Fantasma 2 (Cian – img derecha)
for ax in axes: ax.axis("off")
```

```
plt.tight_layout()
plt.show()
```

Medición de disparidad en píxeles

Haga zoom en el anaglifo de arriba (o en las imágenes separadas) y busque un punto distintivo (esquina, borde, letra) en cada objeto. Anote la **coordenada X** del borde rojo y del borde cian del mismo rasgo.

Luego ingrese las coordenadas en la celda siguiente.

In [160…

```
import plotly.express as px

fig = px.imshow(anaglyph_u8, title="Haga zoom y pase el cursor para leer coorden
fig.update_layout(width=1200, height=700)
fig.write_html("anaglyph_interactive.html")
print("Abra el archivo 'anaglyph_interactive.html' en su navegador.")
print("Pase el cursor sobre la imagen para ver x, y.")
```

```
Abra el archivo 'anaglyph_interactive.html' en su navegador.
Pase el cursor sobre la imagen para ver x, y.
```

In [162…

```
B_x1 = 48
B_x2 = 214

A_x1 = 383
A_x2 = 813

C_x1 = 347
C_x2 = 423

dist_real = ["~1.5 m", "~3 m", "~6 m"]

objects = ["A (Cercano)", "B (Medio)", "C (Fondo)"]
x1_vals = [A_x1, B_x1, C_x1]
x2_vals = [A_x2, B_x2, C_x2]
disparities = [abs(x1 - x2) for x1, x2 in zip(x1_vals, x2_vals)]

print(f"{'Objeto':<15} {'Dist. Real Est.':<18} {'Coord X Fantasma 1':<22} {'Coor
print("=" * 95)
for i in range(3):
    print(f"{objects[i]:<15} {dist_real[i]:<18} {x1_vals[i]:<22.1f} {x2_vals[i]:

fig, ax = plt.subplots(figsize=(12, 3))
```

```
ax.axis("off")

table_data = []
for i in range(3):
    table_data.append([
        objects[i],
        dist_real[i],
        f"{x1_vals[i]:.1f}",
        f"{x2_vals[i]:.1f}",
        f"{disparities[i]:.1f} px"
    ])

col_labels = ["Objeto", "Distancia Real\nEstimada", "Coordenada X\nFantasma 1",
table = ax.table(cellText=table_data, colLabels=col_labels, loc="center", cellLo
table.auto_set_font_size(False)
table.set_fontsize(11)
table.scale(1.2, 1.8)
plt.title("Tabla de Disparidades – Task 2.2", fontsize=14, pad=20)
plt.tight_layout()
plt.show()
```

```
Objeto          Dist. Real Est.    Coord X Fantasma 1     Coord X Fantasma 2
Disparidad |x1-x2|
================================================================================
==============
A (Cercano)     ~1.5 m             383.0                  813.0
430.0 px
B (Medio)       ~3 m               48.0                   214.0
166.0 px
C (Fondo)       ~6 m               347.0                  423.0
76.0 px
```

Tabla de Disparidades – Task 2.2

Objeto	Distancia Real Estimada	Coordenada X Fantasma 1	Coordenada X Fantasma 2	Disparidad (|x1-x2|) en píxeles
A (Cercano)	~1.5 m	383.0	813.0	430.0 px
B (Medio)	~3 m	48.0	214.0	166.0 px
C (Fondo)	~6 m	347.0	423.0	76.0 px

Task 3

Construya un reporte, donde responda a las siguientes preguntas basándose exclusivamente en sus datos:

Pregunta 1

Sí, existe una tendencia inversa clara: a mayor distancia, menor disparidad.

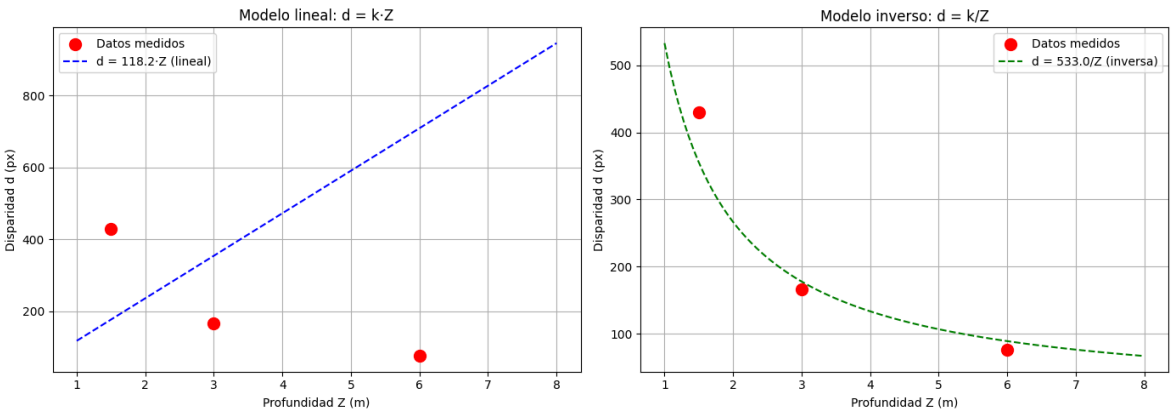
Objeto	Distancia	Disparidad
A (Cercano)	~1.5 m	430 px
B (Medio)	~3 m	166 px
C (Fondo)	~6 m	76 px

Al duplicar la distancia de 1.5 m a 3 m, la disparidad cae por un factor de ~2.6×. De 3 m a 6 m, cae por ~2.2×. Los objetos cercanos se desplazan más entre vistas, los lejanos se alinean mejor.

In [174…

```
Z = np.array([1.5, 3.0, 6.0])
d = np.array([430.0, 166.0, 76.0])

k_lineal = d / Z
k_inversa = d * Z

print("Verificación del modelo lineal (d = k·Z):")
for i in range(3):
    print(f"  Z={Z[i]:.1f} m, d={d[i]:.0f} px → k = d/Z = {k_lineal[i]:.1f}")

print(f"\nVerificación del modelo inverso (d = k/Z):")
for i in range(3):
    print(f"  Z={Z[i]:.1f} m, d={d[i]:.0f} px → k = d·Z = {k_inversa[i]:.1f}")

Z_fit = np.linspace(1.0, 8.0, 100)
k_inv_avg = np.mean(k_inversa)
k_lin_avg = np.mean(k_lineal)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].scatter(Z, d, color='red', s=100, zorder=5, label='Datos medidos')
axes[0].plot(Z_fit, k_lin_avg * Z_fit, 'b--', label=f'd = {k_lin_avg:.1f}·Z (lin
axes[0].set_xlabel('Profundidad Z (m)')
axes[0].set_ylabel('Disparidad d (px)')
axes[0].set_title('Modelo lineal: d = k·Z')
axes[0].legend()
axes[0].grid(True)

axes[1].scatter(Z, d, color='red', s=100, zorder=5, label='Datos medidos')
axes[1].plot(Z_fit, k_inv_avg / Z_fit, 'g--', label=f'd = {k_inv_avg:.1f}/Z (inv
axes[1].set_xlabel('Profundidad Z (m)')
axes[1].set_ylabel('Disparidad d (px)')
axes[1].set_title('Modelo inverso: d = k/Z')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()

residuos_lin = d - k_lin_avg * Z
residuos_inv = d - k_inv_avg / Z
print(f"\nError cuadrático medio (lineal):  {np.mean(residuos_lin**2):.1f}")
print(f"Error cuadrático medio (inversa): {np.mean(residuos_inv**2):.1f}")
```

```
Verificación del modelo lineal (d = k·Z):
  Z=1.5 m, d=430 px → k = d/Z = 286.7
  Z=3.0 m, d=166 px → k = d/Z = 55.3
  Z=6.0 m, d=76 px → k = d/Z = 12.7

Verificación del modelo inverso (d = k/Z):
  Z=1.5 m, d=430 px → k = d·Z = 645.0
  Z=3.0 m, d=166 px → k = d·Z = 498.0
  Z=6.0 m, d=76 px → k = d·Z = 456.0
```




```
Error cuadrático medio (lineal):  166848.9
Error cuadrático medio (inversa): 1958.6
```

Pregunta 2

La relación se ajusta al **modelo inverso d = k/Z**.

- **Lineal (d = k·Z):** k = d/Z da valores muy distintos (286.7, 55.3, 12.7) → no es constante.
- **Inverso (d = k/Z):** k = d·Z da valores más estables (645, 498, 456) → mejor ajuste.

Esto es consistente con la ecuación de estereoscopía: **d = (f·B) / Z**.

Pregunta 3

La homografía solo alinea correctamente escenas planares o con rotación pura de cámara. Nuestras fotos tienen traslación lateral y objetos a distintas profundidades, por lo que RANSAC alinea el plano dominante (pared del fondo) y los objetos cercanos quedan desplazados.

La información "escondida" en ese fallo es la **profundidad (Z)**. La disparidad entre los fantasmas rojo y cian codifica directamente la distancia de cada objeto a la cámara (d = k/Z), que es el principio fundamental de la estereoscopía.